CineExplorer

RACZKOWSKI Dawid (s200735), BUI Hoang Linh (s2503303), DINH Duy Vu (s2401627)

Knowledge representation and reasoning, DEBRUYNE Christophe, University of Liège

April 17, 2026

Abstract

This report covers Milestones 2 and 3 of the CineExplorer project. In Milestone 2, an ontology is designed to capture the main concepts of the audiovisual domain and their relationships. The ontology is guided by competency questions and implemented in OWL 2 DL, serialized in Turtle syntax, and edited in Protégé. Key design decisions include the introduction of `CreativeWork` as a central abstraction, the use of a reified `Participation` class to represent contextual relations between persons and works, and the use of inverse properties to support graph traversal.

In Milestone 3, the relational database is annotated using an R2RML mapping that generates an RDF knowledge graph conforming to the ontology. The mapping covers the main entity types and applies a stable IRI strategy based on IMDb identifiers.

Contents

1 Introduction

CineExplorer is an intelligent exploration platform designed to integrate and annotate autonomous information systems via an ontology. To move beyond traditional film databases, the system requires a Knowledge Graph (KG) architecture that reveals the complex connections within the world of cinema. While drawing primarily on the Internet Movie Database (IMDb) [3] relational dataset, the system is designed to integrate and align with external data sources such as Wikidata [8] to resolve ambiguities and enrich entity profiles with missing production and contextual attributes.

CineExplorer offers a dynamic interface that provides detailed information on film and actor profiles, with navigation directly controlled by the KG. Thanks to the ontology developed, CineExplorer will offer graph-based exploration capabilities, structured around three main areas:

1. CineExplorer combines data from IMDb with Wikidata to resolve semantic ambiguities. This enhancement process allows contextual and production attributes that are absent from the initial dataset to be integrated, thereby ensuring the completeness of the entity set.

2. Beyond simple searches, users will be able to discover the link between actors based on the concept of Bacon Numbers. Using the graph, the application will calculate semantic paths to answer complex questions such as determining how many films separate two actors.

3. The interface will be generated by the graph. Navigation interfaces will be dynamic and based on the ontology developed.

This report covers two milestones. Milestone 2 describes the design and implementation of the ontology, including the main design decisions, the adopted OWL 2 profile, and the engineering process. Milestone 3 describes how the relational database is annotated using an R2RML mapping to produce an RDF knowledge graph conforming to the ontology.

2 Dataset Description

In this project, the IMDb non-commercial dataset [3] is used as the upstream data source. IMDb provides a core subset of its data for personal and non-commercial use, intended to be downloaded and stored locally, and subject to IMDb's terms and conditions [3, 2].

For this project, the dataset is provided in a relational form as a sampled, normalized subset derived from the official IMDb TSV files. The transformation and sampling process follows the approach described by northCoder [6, 7]. The raw IMDb files are processed and rearranged into a schema closer to third normal form, and a smaller sample is extracted for testing and demonstration. The resulting relational database models titles (movies, series, episodes), talent, credited participation, genres, alternative titles (with language and region metadata), and episode hierarchies, which align with the requirements of our Semantic Movie and Series Explorer application.

We note that the scripts and relational sample packaging are distributed under an open-source license in the referenced repository, while the underlying IMDb data remains subject to IMDb's non-commercial usage conditions [2, 7]. This data is also made available under the MIT license, which allows for unrestricted use, copying, modification, merging, publishing, distribution, sublicensing, and/or selling of copies of the licensed elements.

3 Towards an Information System

This section presents the conceptual Entity-Relationship Diagram (ERD), the key modeling assumptions and cardinalities, and how the provided sample is used to populate the database.

3.1 Entity-relationship Diagram

Figure 3.1 illustrates the ERD of the IMDb. The ERD models the main concepts required to represent entertainment titles, people involved in them, their classifications, their alternative regional or linguistic titles, and the episode hierarchy of series.

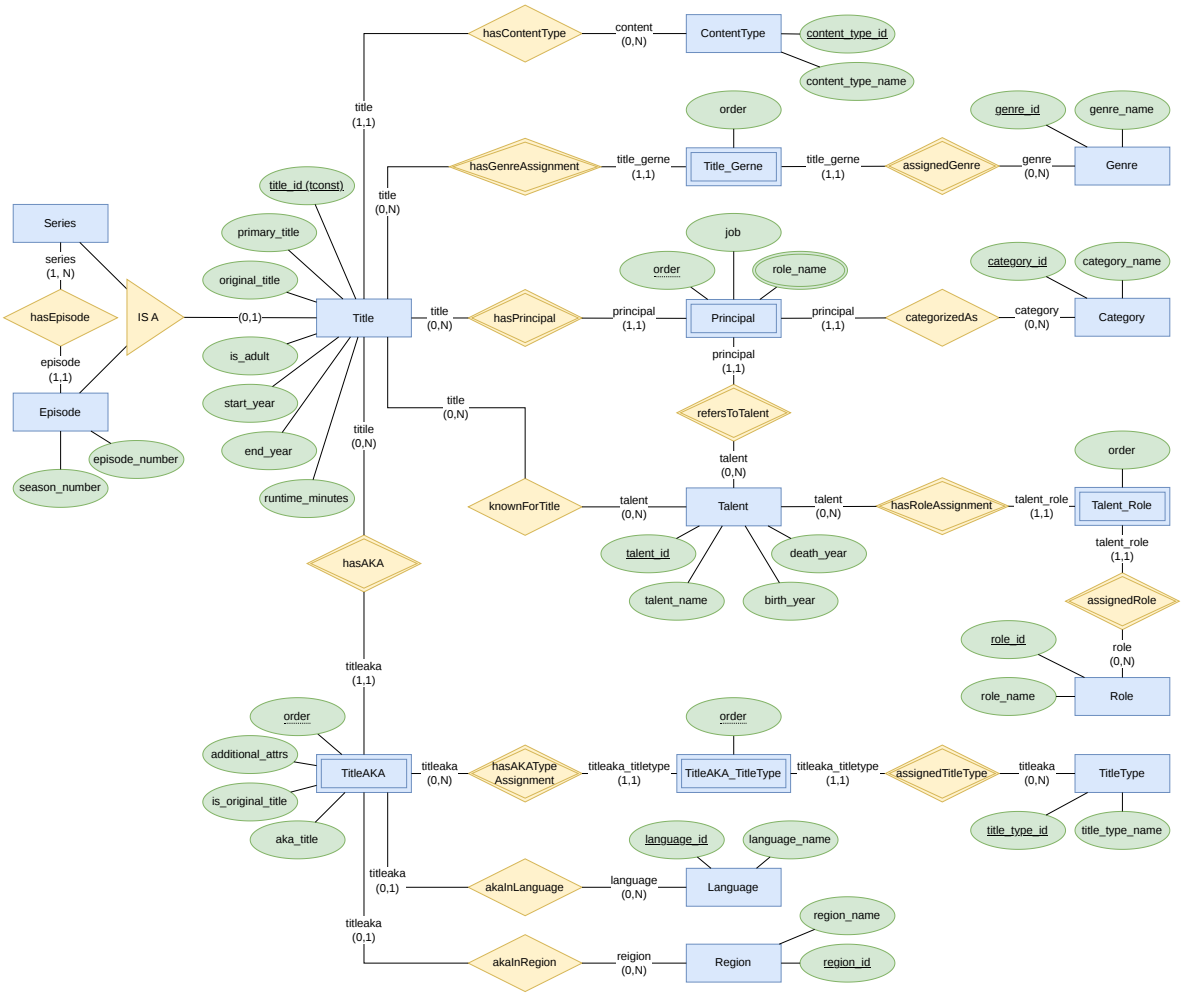

Figure 3.1: ERD for IMDb.

The central entity set is `Title`, representing an audiovisual work. It stores core descriptive attributes such as identifier, primary and original title, adult flag, years, and runtime. Each `Title` is classified by exactly one `ContentType` (e.g., movie, short, TV series). An `IS-A` specialization is only introduced for `Series` and `Episode` because episodic content requires additional structure that cannot be captured by classification alone: a `Series` can have episodes, while an `Episode` belongs to exactly one series and is positioned by `season_number` and `episode_number`. The specialization is partial, meaning a `Title` may be neither a series nor an episode; in that case, it is still represented as a `Title` and classified through `ContentType`. Genres are modeled using the weak entity set `Title_Genre` between `Title` and `Genre`.

The entity set `Talent` represents people involved in titles (cast and crew). To model credits, we use the weak entity set `Principal` connecting `Title`, `Talent`, and `Category` (e.g., actor, director). This design is required because participation is not a simple many-to-many relationship. It carries additional attributes, namely `order`, `job`, and `role_names`. In addition, professions of a person are represented through `Talent_Role`, a weak entity set between `Talent` and `Role`, again preserving `order` where present.

Alternative and localized titles are modeled through the weak entity set `TitleAKA`, which depends on `Title` via the identifying relationship `hasAKA`. TitleAKA is identified by its owning Title and a local discriminator `order`. It stores the alternative title text and related metadata. TitleAKA may optionally be linked to `Language` and `Region`, enabling language-specific and region-specific exploration. TitleAKA may also be classified by `TitleType` through the weak entity set `TitleAKA_TitleType`.

Episodic content is represented through the relationship `hasEpisode` between `Series` and `Episode`,

with `season_number` and `episode_number` supporting ordered navigation.

A more detailed overview of the modeling assumptions, cardinalities, and attribute domains used in the relational schema is provided in Appendix C.

3.2 Meaningful Examples

The northCoder repository [7] provides multiple `.csv` files containing a sampled dataset (including 174 titles) together with reference tables (genres, categories, languages, regions, etc.). We load these files into the SQL relational database using the provided `.sql` script and verify successful population using row-count checks per table.

3.3 SQL Implementation

The SQL implementation follows the ERD described previously and the repository northCoder [7]. The scripts from this repository were used as the initial implementation and then adapted to better match our conceptual model.

First, several foreign key constraints were missing in the original scripts. These constraints were added to enforce the relationships defined in the ERD. During this process, an inconsistency in the dataset was discovered: some records in the `title_principal` table referenced `talent_id` values that were not present in the `talent` table. Since this prevented the creation of the foreign key constraint, these invalid records were removed in order to maintain referential integrity.

Second, the structure of the principal data was not fully normalized. In the original dataset, the attribute `role_names` may contain multiple values stored as a comma-separated string. This violates the first normal form (1NF). To address this issue, the schema was normalized by splitting the original structure into two tables: `title_principal`, which stores the participation of a talent in a title, and `principal_role`, which stores the individual role names associated with each participation record.

These modifications ensure that the relational schema is consistent with the ERD and respects normalization and referential integrity constraints.

4 Ontology Engineering

The CineExplorer ontology provides a conceptual model for representing creative works and the persons involved in them. It is designed to support the annotation of the relational database and the construction of an RDF knowledge graph.

4.1 Scope

The ontology supports three main application directions. It enables actor connectivity analysis, including Bacon-style path exploration between persons connected through shared creative works. This functionality is enabled by the structure of the RDF graph, where persons are linked to works through participation relations.

It also provides a conceptual structure for aligning local entities with external resources such as Wikidata [8]. In addition, it supports graph-based exploration of related entities through navigation across persons, creative works, genres, and participation relations.

The ontology therefore covers the concepts required for these use cases: `CreativeWork` instances (`Film`, `Series`, and `Episode`), `Person`, `Genre`, descriptive attributes such as language and region, and contextual participation relations. This structure also supports interface-level exploration driven by the knowledge graph.

4.2 Concept Organization

The ontology is organized into classes, object properties, and data properties. This separation maintains a clear distinction between entities, relations, and literal attributes.

The ontology structure is visualized in Figure 4.2 using WebVOWL [4].

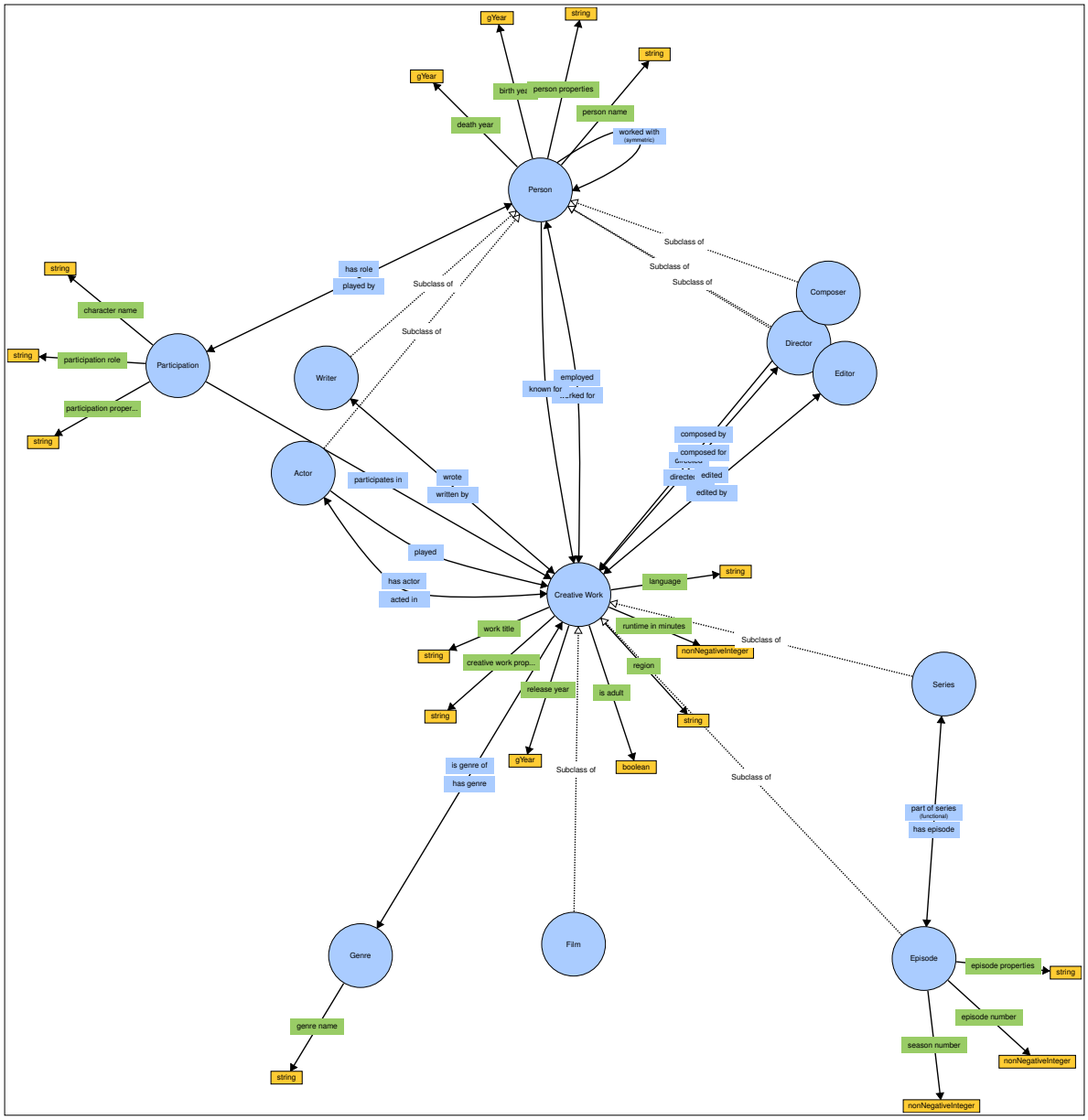

Figure 4.2: CineExplorer ontology.

4.2.1 Classes

The central class is `CreativeWork`, which represents a work independently of its textual representation in the dataset. This abstraction separates the notion of a work from its titles.

`CreativeWork` is specialized into `Film`, `Series`, and `Episode`. These subclasses are declared mutually disjoint. The class `Person` represents individuals involved in productions and is specialized into `Actor`, `Director`, `Writer`, `Editor`, and `Composer`. These subclasses are not declared disjoint, as multiple roles may be associated with the same individual.

Relationship to the source data. The ontology is informed by the structure of the underlying relational schema. Class and property names correspond to elements present in the dataset, which simplifies the mapping and ensures interpretability. At the same time, the ontology introduces abstraction layers that are not present in the database. In particular, the use of `CreativeWork` instead of `Title` and the introduction of `Participation` reflect conceptual modeling decisions rather than direct schema replication.

The `Participation` class. The class `Participation` reifies the relation between a `Person` and a `CreativeWork`. This design allows contextual attributes such as role and character name to be associated with the relation itself.

`Participation` is declared disjoint from `CreativeWork`, `Person`, and `Genre`. It is connected to exactly one `Person` through `playedBy` and to exactly one `CreativeWork` through `participatesIn`. These axioms define the intended semantics of participation under the open-world assumption and do not act as database integrity constraints.

`Episode` is associated with a `Series` through `partOfSeries`, which is declared functional.

4.2.2 Object Properties

Object properties define relations between entities. These include relations between works and genres, relations between episodes and series, and relations between persons and works through `Participation`.

The properties `workedFor` and `employed` define a general relation between `Person` and `CreativeWork`. More specific properties such as `actedIn`, `directed`, `wrote`, `edited`, and `composedFor` are defined as sub-properties. Each property is associated with an inverse to support bidirectional navigation.

The property `workedWith` represents a relation between persons derived from shared participation in the same creative work. It is primarily a traversal-oriented relation and can be derived from participation links, but may also be materialized to simplify querying.

The property `sameAsExternal` links local resources to external identifiers. No cardinality restriction is imposed on this property.

4.2.3 Data Properties

Data properties associate entities with literal values. `Person` is described by attributes such as name and life dates. `CreativeWork` is described by attributes such as title, release year, runtime, and content flags. `Participation` includes attributes describing roles and character information. `Episode` includes season and episode numbers.

Language and region as data properties. Language and region are represented as literal attributes of `CreativeWork`. This choice keeps the ontology simple and consistent with their descriptive role in the dataset.

4.3 OWL Profile

The ontology is implemented in OWL 2 DL [5]. This follows directly from the constructs used in the ontology.

The ontology includes qualified cardinality restrictions, functional properties, inverse properties, and disjointness axioms. These features are not fully supported in OWL 2 EL or OWL 2 QL, which places the ontology within OWL 2 DL.

Verifying OWL 2 DL compliance. The ontology was checked using the HermiT reasoner in Protégé. The ontology was successfully classified without errors, indicating that no OWL 2 Full constructs are used.

Cardinality axioms vs. database constraints. OWL cardinality axioms and relational constraints serve different purposes. Database constraints enforce data validity under a closed-world assumption. OWL axioms define class semantics under an open-world assumption. In this ontology, cardinality axioms describe intended meaning rather than enforce completeness of data.

4.4 Namespace and Documentation

A single namespace is used: `https://example.org/cineexplorer/ontology#`. Classes and properties are annotated with `rdfs:label` and `rdfs:comment`. Naming follows standard conventions, with UpperCamelCase for classes and lowerCamelCase for properties.

4.5 Encountered Problems

A main challenge was handling directionality of relations. Explicit inverse properties were introduced to support bidirectional navigation and improve graph usability.

Another issue concerned redundant axioms. Cardinality expressions of the form `min 0` were removed, as they do not contribute semantic information.

5 Annotating the Information System

This section describes how the relational database is mapped to RDF triples using R2RML [1], producing an instance of the CineExplorer knowledge graph conforming to the ontology described in the previous section.

5.1 Approach

R2RML (RDB to RDF Mapping Language) [1] is used as the mapping language. R2RML is a W3C recommendation that defines how relational tables and SQL queries can be mapped to RDF graphs. The mapping file `cineexplorer_mapping.ttl` is serialized in Turtle syntax and processed by the `r2rml.jar` tool used in the lab sessions. The generated RDF output is written to `output/cineexplorer_kg.ttl` in Turtle format.

The mapping is executed with the following command from the project root:

```
java -jar tools/r2rml/r2rml.jar mapping/mapping.properties
```

where `mapping.properties` provides the JDBC connection string, credentials, input mapping file path, and output file path.

The mapping strategy follows the conceptual structure of the ontology rather than directly mirroring the table structure. In particular, the mapping is organized around the main ontology entities (`CreativeWork`, `Person`, `Participation`, and `Genre`) and around a small number of additional triples maps used to enrich these entities with structural or descriptive information.

5.2 IRI Strategy

A consistent IRI strategy was defined to ensure that each entity has a stable identifier. IRIs are based on the IMDb identifiers already present in the database, rather than on surrogate numeric keys generated by our schema. This ensures stable identifiers and reuse of widely recognized identifiers

The main IRI templates used are:

- **Creative works** (`Film`, `Series`, `Episode`):
  `https://example.org/cineexplorer/title/{TITLE_ID}`
  where `TITLE_ID` is the IMDb `tconst` (e.g., `tt0000001`).

- **Persons**:
  `https://example.org/cineexplorer/person/{TALENT_ID}`
  where `TALENT_ID` is the IMDb `nconst` (e.g., `nm0000001`).
- **Genres**:
  `https://example.org/cineexplorer/genre/{GENRE_ID}`
  where `GENRE_ID` is the internal genre identifier.
- **Participation nodes**:
  `https://example.org/cineexplorer/participation/{TITLE_ID}/{TALENT_ID}/{ORD}`
  where `ORD` is the credit order from the `title_principal` table.

For genres, an internal identifier is used because the source data does not provide IMDb-style identifiers for genre instances. The participation IRI is composite because a participation record is context-dependent: it is identified by the work, the person, and the credit order.

This strategy avoids artificial blank nodes for the main entities and ensures that resources can be referenced consistently across triples maps.

5.3 Mapping Structure

The mapping is organized by entity type and by the role played by each triples map in the generation of the RDF graph.

Creative works. Three triples maps generate the three subclasses of `CreativeWork`. The distinction between `Film` and `Series` is based on the `CONTENT_TYPE_ID` column in the `title` table. `Film` includes non-episodic works such as movies, short films, TV movies, TV shorts, TV specials, and videos. `Series` includes TV series and TV mini-series. In both cases, titles that appear in `title_episode` are excluded in order to avoid misclassifying episodic content.

The `Episode` triples map is based directly on the join between `title` and `title_episode`. This is more robust than relying only on content type, because episodic membership is explicitly represented in the relational schema. The mapping generates the relation `partOfSeries` together with `seasonNumber` and `episodeNumber`.

Persons. All rows from the `talent` table are mapped to `ce:Person` with their descriptive data properties. Role-specific subclass membership (`Actor`, `Director`, `Writer`, `Composer`, `Editor`) is handled through separate triples maps based on joins between `talent_role` and `role`. Since these maps share the same IRI template, the resulting triples are merged on the same subject, allowing one person to belong to multiple role-specific subclasses.

Participation. The `Participation` triples map selects from `title_principal` joined with `category` and generates one participation node per credit record. Each node is linked to its person via `ce:playedBy` and to its work via `ce:participatesIn`. The category name is stored as `ce:participationRole`, while the free-text job description is stored as `ce:participationProperties`.

Character names, which were normalized into the separate `principal_role` table during Milestone 1, are handled by a dedicated triples map targeting the same participation IRI template. This adds `ce:characterName` triples without duplicating participation nodes.

Genre links. Genre instances are mapped from the `genre` table. The relation between titles and genres is generated from `title_genre` through `ce:hasGenre`. This keeps genre resources separate while preserving the many-to-many relation present in the database.

Known-for links. The `talent_title` table, which records notable works associated with a person, is mapped to `ce:knownFor` links from person IRIs to title IRIs.

Additional traversal-oriented properties. Besides the participation-centered representation, the mapping also materializes several direct properties between persons and creative works, such as `workedFor` and `employed`, together with more specific sub-properties such as `actedIn`, `directed`, `wrote`, `edited`, `composedFor`, and `played`. These properties are redundant from a purely normalization-oriented point of view, since they can be recovered through `Participation`, but they make graph traversal and query writing more convenient.

Similarly, `workedWith` is included as a traversal-oriented relation between persons who are connected through participation in the same creative work. This relation is useful for graph exploration use cases such as Bacon-style path analysis.

Language and region metadata. Language and region information is derived from `title_aka` together with the lookup tables `language` and `region`. Rather than introducing standalone resources for these entities, the mapping attaches them as descriptive literals to `CreativeWork` instances through the datatype properties `ce:language` and `ce:region`. This is consistent with the ontology, where language and region are modeled as descriptive attributes rather than as independent conceptual classes.

6 Conclusion

This report covered Milestones 2 and 3 of the CineExplorer project.

In Milestone 2, we designed an OWL 2 DL ontology that abstracts the relational IMDb dataset into a meaningful semantic model. The main design decisions were the introduction of `CreativeWork` as a domain abstraction over the database `Title` concept, the reification of participation through an explicit `Participation` class, and the systematic use of inverse properties to support bidirectional graph traversal. The ontology was verified for OWL 2 DL compliance using the HermiT reasoner in Protégé.

In Milestone 3, we produced an R2RML mapping that generates an RDF knowledge graph from the MySQL database. The mapping applies a stable IRI strategy based on IMDb identifiers and covers all major entity types, including films, series, episodes, persons, genres, and participation records. The mapping was executed successfully and the output knowledge graph conforms to the ontology.

The resulting ontology and knowledge graph provide a foundation for further work, including SPARQL querying, external data integration, and graph-based analytics such as Bacon number computation.

7 Member Contributions

Although each member focused on specific tasks (Table 7.1), we worked collaboratively during the review and editing phases.

Table 7.1: Member contributions.

Name	Contribution
Dawid	Ontology concept organization and class hierarchy; OWL cardinality restrictions; ontology implementation in Turtle and Protégé; R2RML mapping for creative works and episodes.
Hoang Linh	Concept naming and design decisions; OWL profile analysis; object property hierarchy and inverse properties; R2RML mapping for persons and participation.
Duy Vu	Application scope and competency questions; ontology annotations and namespace; IRI strategy; R2RML mapping for genres and known-for links; report writing and coordination.

References

[1] R. Cyganiak, A. Das, and J. Sequeda. R2RML: RDB to RDF Mapping Language. `https://www.w3.org/TR/r2rml/`, 2012. W3C Recommendation, 27 September 2012.

[2] IMDb. Can i use imdb data in my software? https://help.imdb.com/article/imdb/general-information/can-i-use-imdb-data-in-my-software/G5JTRESSHJBBHTGX. Accessed: 2026-02-22.

[3] IMDb. Imdb non-commercial datasets. https://developer.imdb.com/non-commercial-datasets/, 2025. Accessed: 2026-02-26.

[4] S. Lohmann, S. Negru, F. Haag, and T. Ertl. Webvowl: Web-based visualization of ontologies. In Proceedings of the 19th International Conference on Knowledge Engineering and Knowledge Management, 2014.

[5] B. Motik, B. C. Grau, I. Horrocks, Z. Wu, A. Fokoue, and C. Lutz. OWL 2 Web Ontology Language: Profiles (Second Edition). https://www.w3.org/TR/owl2-profiles/, 2012. W3C Recommendation, 11 December 2012.

[6] northCoder. Using imdb as a test data set. https://northcoder.com/post/using-imdb-as-test-data-set, 2019. Accessed: 2026-02-22.

[7] northCoder repo. Sample imdb data for relational databases. for testing/demo purposes only. https://github.com/northcoder-repo/relational-sample-imdb-data, 2019. Accessed: 2026-02-22.

[8] Wikidata. Wikidata. https://www.wikidata.org, 2023. Accessed: 2026-03-05.

A Appendix: The use of AI

We used AI tools in a partially supportive way. ChatGPT was employed to explain references, summarize abstracts, and suggest content reorganization for better flow. DeepL and ChatGPT were used for grammar and language correction

B Competency Questions

1. What is the shortest collaboration path between two actors or other persons involved in audiovisual productions?
2. Through which creative works are two persons connected?
3. Which persons are central in the collaboration graph?
4. Which local entities can be aligned with external resources such as Wikidata?
5. What additional contextual information can be added through external enrichment?
6. Which episodes belong to a given series?
7. Which genres, languages, and regions are associated with a given creative work?
8. Which related entities can be discovered when navigating from a film, person, or series in the knowledge graph?

C Additional Information on the Relational Model

C.1 Modeling Assumptions and Cardinalities

Table C.2 summarizes the main modeling assumptions and min-max cardinalities used in the ERD.

Table C.2: Key ERD assumptions and cardinalities.

Element	Assumption / cardinality (min,max)
Title - ContentType	Each Title has exactly one ContentType: Title (1,1) to ContentType (0,N).
Title - Genre (via Title_Genre)	A Title may have zero or more genres and a Genre may classify zero or more titles: Title (0,N) to Genre (0,N). Title_Genre stores `order`.
Principal (Title, Talent, Category)	Each Principal record refers to exactly one Title, one Talent, and one Category. Title (0,N) to Principal (1,1); Talent (0,N) to Principal (1,1); Category (0,N) to Principal (1,1). Principal stores `order`, `job`, `role_names`.
TitleAKA as weak entity	TitleAKA is a weak entity identified by Title plus local `order`. Identifying relationship: Title (0,N) to TitleAKA (1,1).
TitleAKA - Language / Region	Language and Region links are optional: TitleAKA (0,1) to Language (0,N); TitleAKA (0,1) to Region (0,N).
Episode and Series specialization	Episode IS-A Title. Series IS-A Title (conceptual). Specialization is partial (not every Title is Episode/Series) and disjoint (Episode and Series do not overlap).
Series - Episode	Each Episode belongs to exactly one Series: Episode (1,1) to Series (1,N). Season and episode numbers support ordering.
Talent - Role (via Talent_Role)	A Talent may have zero or more roles, and a Role may be linked to zero or more talents: Talent (0,N) to Role (0,N). Talent_Role stores `order`.

C.2 Attribute Domains

The attribute domains are inferred below based on the provided dataset [6, 7] and the information from IMDb [3].

- **Title**
  - dom(Title.title_id) = alphanumeric string
  - dom(Title.primary_title) = non-empty string
  - dom(Title.original_title) = non-empty string
  - dom(Title.is_adult) = $\{0, 1\}$
  - dom(Title.start_year) = $\mathbb{N} \cup \{null\}$
  - dom(Title.end_year) = $\mathbb{N} \cup \{null\}$
  - dom(Title.runtime_minutes) = $\mathbb{N}^+ \cup \{null\}$
- **ContentType**
  - dom(ContentType.content_type_id) = $\mathbb{N}^+$
  - dom(ContentType.content_type_name) = non-empty string
- **Genre**
  - dom(Genre.genre_id) = $\mathbb{N}^+$
  - dom(Genre.genre_name) = non-empty string
- **Title_Genre** (weak entity)
  - dom(Title_Genre.order) = $\mathbb{N}^+$
- **Talent**
  - dom(Talent.talent_id) = alphanumeric string
  - dom(Talent.talent_name) = non-empty string
  - dom(Talent.birth_year) = $\mathbb{N} \cup \{null\}$
  - dom(Talent.death_year) = $\mathbb{N} \cup \{null\}$

- **Category**
  - dom(Category.category_id) = $\mathbb{N}^+$
  - dom(Category.category_name) = non-empty string
- **Principal** (weak entity)
  - dom(Principal.order) = $\mathbb{N}^+$
  - dom(Principal.job) = string $\cup\{null\}$
  - dom(Principal.role_name) = string $\cup\{null\}$
- **Role**
  - dom(Role.role_id) = $\mathbb{N}^+$
  - dom(Role.role_name) = non-empty string
- **Talent_Role** (weak entity)
  - dom(Talent_Role.order) = $\mathbb{N}^+$
- **TitleAKA** (weak entity)
  - dom(TitleAKA.order) = $\mathbb{N}^+$
  - dom(TitleAKA.aka_title) = non-empty string
  - dom(TitleAKA.additional_attrs) = string $\cup\{null\}$
  - dom(TitleAKA.is_original_title) = $\{0, 1\}$
- **Language**
  - dom(Language.language_id) = alphanumeric string (language code)
  - dom(Language.language_name) = string
- **Region**
  - dom(Region.region_id) = alphanumeric string (region code)
  - dom(Region.region_name) = string
- **TitleType**
  - dom(TitleType.title_type_id) = $\mathbb{N}^+$
  - dom(TitleType.title_type_name) = non-empty string
- **TitleAKA_TitleType** (weak entity)
  - dom(TitleAKA_TitleType.order) = $\mathbb{N}^+$
- **Episode** (subtype of Title)
  - dom(Episode.season_number) = $\mathbb{N}^+ \cup \{null\}$
  - dom(Episode.episode_number) = $\mathbb{N}^+ \cup \{null\}$

C.3 Description of Non-visible Integrity Constraints

- If both `start_year` and `end_year` are present for a `Title`, then `start_year` $\leq$ `end_year`.
- If both `birth_year` and `death_year` are present for a `Talent`, then `birth_year` $\leq$ `death_year`.